SECURITY FINDINGS DOCUMENTATION

NodeGoat Application — Remediation Tracking

Student Name	Fatima Haq
Course	Cybersecurity
Project Title	Securing the NodeGoat Application
Date	June 26, 2026
Total Findings	4
Fixed	3
Partially Fixed	1

Finding 1: Lack of Input Validation — MEDIUM

Description

The application did not properly validate user inputs during user registration. Invalid or malicious input could be submitted to the server.

Risk

Attackers could provide malformed input, leading to unexpected behavior, invalid data storage, or potential injection attacks.

Evidence

The application accepted invalid email formats before implementing validation.

Remediation

The validator library was implemented to perform server-side validation.

```
const validator = require("validator");
if (!validator.isEmail(email)) {
    errors.emailError = "Invalid email address";
    return false;
}
```

Status

Fixed

Finding 2: Insecure Password Storage — HIGH

Description

User passwords were originally stored in plain text within the MongoDB database.

Risk

If the database is compromised, attackers can directly read all user passwords.

Evidence

Passwords were previously stored as plain text values. Example:

```
password: "mypassword123"
```

Remediation

Passwords are now hashed using bcrypt before storage.

```
password: bcrypt.hashSync(password, bcrypt.genSaltSync())
```

Status

Fixed

Finding 3: Missing Token-Based Authentication — MEDIUM

Description

The application relied only on session-based authentication and did not generate authentication tokens.

Risk

Lack of token-based authentication reduces flexibility and security for authenticated sessions.

Remediation

JWT authentication was implemented.

```
const token = jwt.sign(
    { id: user._id },
    "your-secret-key",
    { expiresIn: "1h" }
);
```

Evidence

After successful login, a JWT token is generated and displayed in the terminal.

Status

Partially Fixed

> **Reviewer note:** Token generation works, but two items remain open: (1) the secret key is hardcoded rather than stored in an environment variable, and (2) no verification middleware was added to protected routes, so issued tokens are not yet being checked anywhere. Marked as Partially Fixed until both are addressed — see Finding 3 remediation plan below.

Finding 4: Missing Secure HTTP Headers — MEDIUM

Description

The application did not implement security headers to protect against common web attacks.

Risk

Missing security headers may expose the application to attacks such as clickjacking, MIME sniffing, and information disclosure.

Remediation

Helmet middleware was implemented.

```
const helmet = require("helmet");
app.use(helmet());
app.disable("x-powered-by");
```

Evidence

Security headers are automatically added to HTTP responses after enabling Helmet.

Status

Fixed

Summary of Findings

Finding	Severity	Status
Lack of Input Validation	Medium	Fixed
Insecure Password Storage	High	Fixed
Missing JWT Authentication	Medium	Partially Fixed
Missing Secure HTTP Headers	Medium	Fixed

Remaining Remediation Plan

To fully close Finding 3 (Missing Token-Based Authentication), the following remains outstanding:

- Move the JWT secret from a hardcoded string to an environment variable (process.env.JWT_SECRET).
- Add jwt.verify() middleware and apply it to protected routes (e.g. /profile, /memos) so issued tokens are actually checked on each request.
- Re-test by attempting to access a protected route without a token and confirming the request is rejected.

Overall Result

Three of the four identified security weaknesses were fully mitigated through input validation, secure password hashing, and Helmet-based HTTP security headers. The fourth finding, JWT authentication, is partially mitigated: tokens are correctly generated at login, but secret management and route-level verification still need to be completed before this control can be marked as fully fixed.

This document was produced as part of a cybersecurity internship training exercise on a deliberately vulnerable application in a local environment.